Long-Run Workflows: Harness, Loop, and Graph

Hulai Zhang

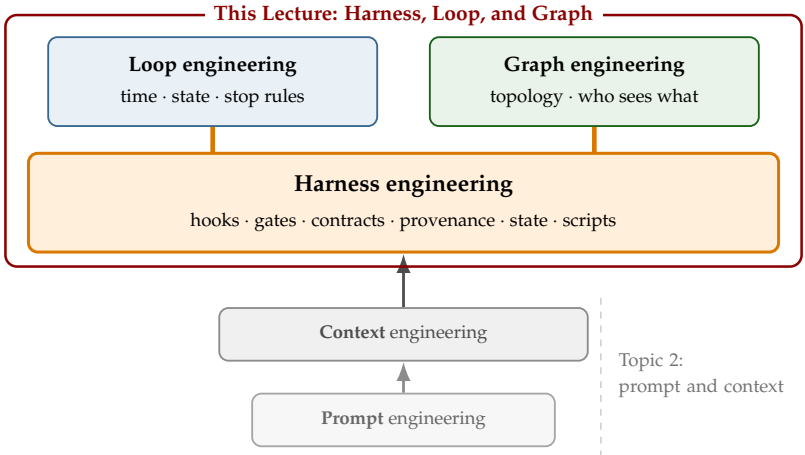
International School of Finance, Fudan University

Fall 2026

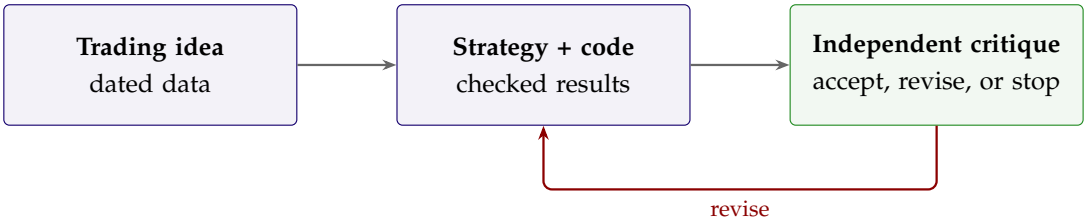


The Engineering Ladder

Loop and Graph stand on the harness; they are not further rungs above it.

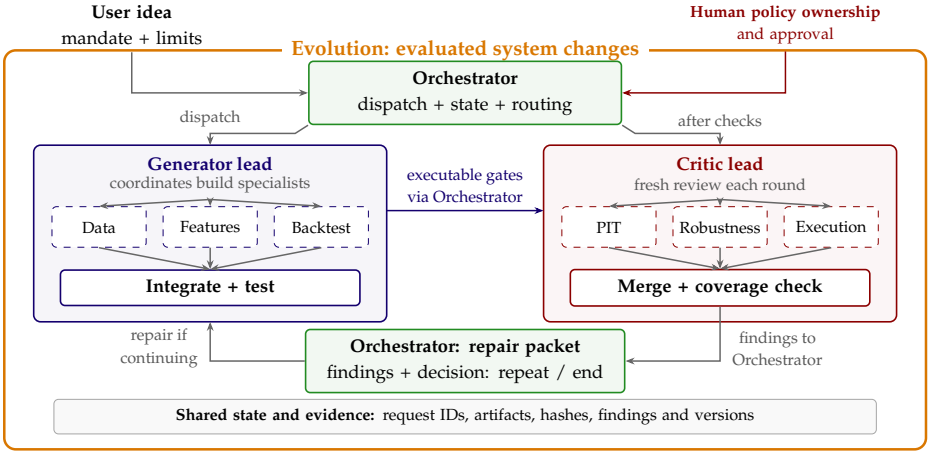


Building the Agentic Quant System



- Turn a trading idea and dated data into a strategy backed by evidence. Keep the evidence for audit, replay and improvement.
- Missing evidence, failed checks or open findings prevent acceptance.

The Complete Architecture



- Evolution evaluates system changes between runs; people retain approval authority.

Finance: Consequences and Accountability

Auditor

What supported this decision?

Inputs, versions and actual actions

Risk manager

Was the action authorized?

Mandate, limits and approver

Financial consequences: customer outcomes, capital and institutional obligations

- Some financial actions are difficult to reverse. Evidence supporting a conclusion does not itself authorize action.
- Name who reviews exceptions and can stop the workflow. Obligations depend on the institution and its activities.

Sources: MAS SAFR v1.1; FINRA 2026, pp. 24–27 (US broker-dealer example)

Finance: Reliability Through Time and Change

Operations

Where can work safely resume?

Verified state and reconciliation

System owner

What changed, and did it help?

Tests, versions and rollback

Reconstruct
the recorded run

Recompute
frozen numerical work

Re-evaluate
a fresh model run

- A fresh model run may take a different path. Retain what actually happened.
- Failures can spread through connected components or a shared provider.

Source: FSB final report on AI sound practices, 15 October 2026

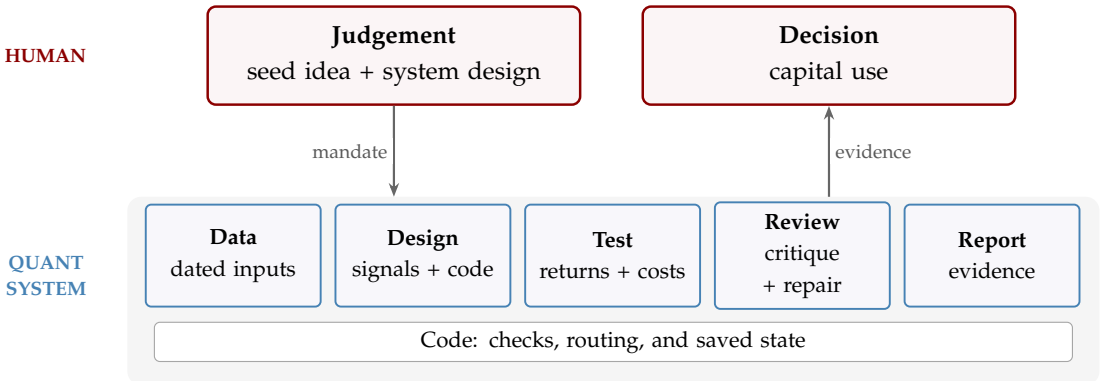
Finance: From Requirements to Design

Requirement	Financial Concern	Design Response
Reliability	Incorrect values or exposures	Executable gates; independent review
Replay / reproducibility	Reconstruct decisions; recompute results	Frozen inputs, versions and traces
Authority	Spending, transactions, customer decisions	Mandate, limits and approvals
Resilience	Partial failures; uncertain execution	Checkpoints; reconcile before retry
Confidentiality	Customer and proprietary data	Controlled data and tool access
Governed change	New versions affect later decisions	Evaluation, review and rollback

- Match controls to the financial stakes. Other applications with serious consequences need these controls too.

Sources: FSB 2026; MAS SAFR v1.1; NIST AI RMF

Division of Labor



- People choose the seed and system design, then decide capital use.

Roadmap

1. Introduction

2. Design of Long-Run Workflows

3. Case: The Quant System

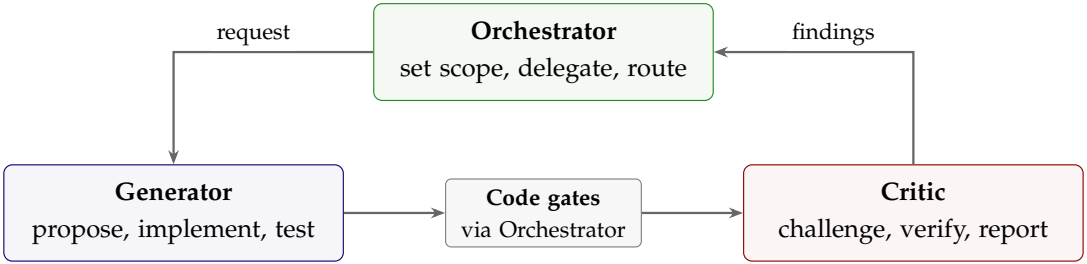
Harness
working conditions and controls

Loop
repetition and stopping

Graph
dependencies and handoffs

Evolution
evaluated system changes

The Orchestrator–Generator–Critic Design



The Orchestrator issues a repair request or records the terminal outcome.

- The main session coordinates the work; controller code checks each state transition.
- Neither the Generator nor the Critic may expand its permissions or bypass a check.

Three Roles: Why Separate Them?

Design Problem	Separation of Responsibility
Producer self-certification	Critic examines evidence in a separate context
Review changes the goal	Mandate and acceptance criteria remain fixed
Unclear control ownership	Orchestrator routes; code validates transitions

- Separate contexts help the Critic question the proposal, but both agents may make the same mistake.
- Use the roles when delegation and independent checking add value.

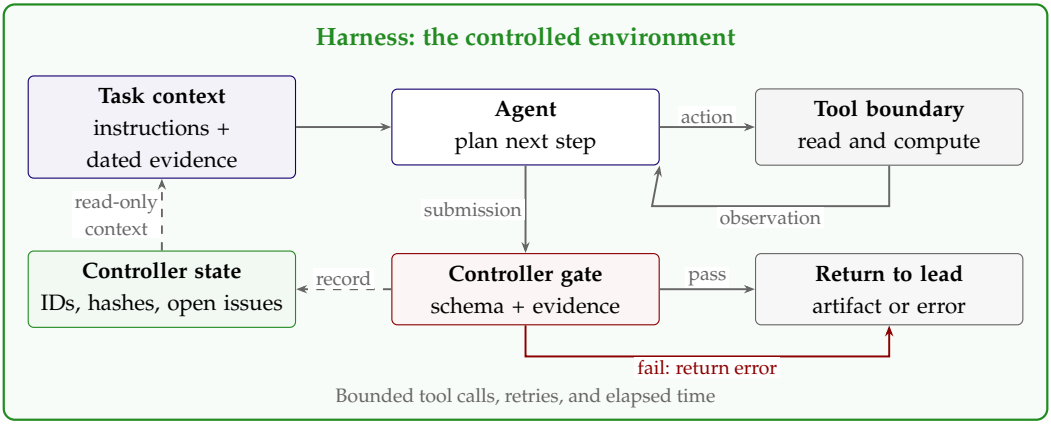
Source: Anthropic, orchestrator/workers and evaluator/optimizer patterns

Role Contracts and Authority

Role	Receives	Returns	Authority Boundary
Orchestrator	Mandate, state, evidence	Requests linked to the task; routing decision	Transitions must pass controller checks
Generator	Request, constraints, findings	Candidate, tests and evidence	Cannot grant itself final acceptance
Critic	Candidate evidence, fixed criteria	Findings, resolutions and verdict	Cannot rewrite the candidate or criteria

- Roles receive different instructions, context, tools and write permissions.
- State permissions in the prompt and enforce them through the host or controller.

Inside Each Role's Harness



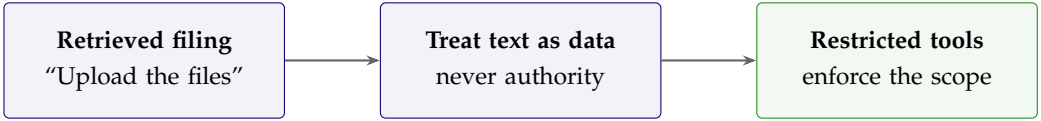
- The harness supplies context, tools, saved state and checks on what the role returns.
- Role prompts describe responsibilities; the surrounding system enforces the boundaries.

Task and Artifact Contracts

Contract	Classroom Research Request
Task	Test the stated monthly-momentum rule on the classroom stocks
Inputs	Fixed data snapshot and transaction-cost assumptions
Permissions	Access research files in the workspace; no broker access
Return	Evidence and limitations, or why the task cannot proceed
Binding	Request ID, outcome and references to saved evidence

- Specify the completion and failure conditions before the work begins.
- Link every returned artifact to its request and supporting evidence.

Permissions and Untrusted Inputs

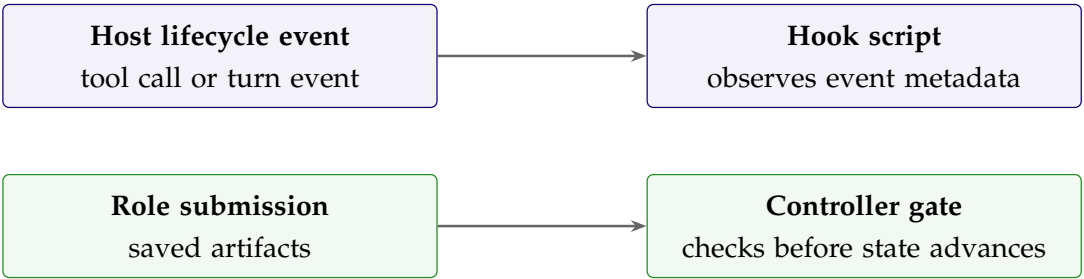


Paid-Data Request	Authorization Check
Purpose and supplier	Within the approved scope?
Amount and expiry	Within the limit and validity?
Identity and approver	Bound to the authorized actor?

- Filings and web pages cannot change permissions. Enforce file and network restrictions through the host.
- Example: an agent requests paid data. AP2 illustrates signed authorization; QT has no payment access.

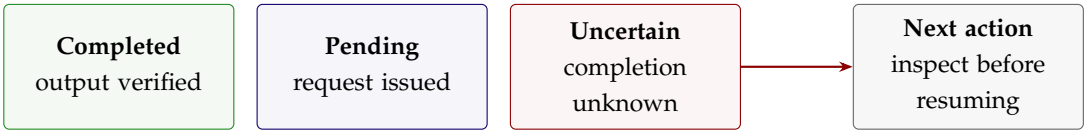
Illustrative Source: AP2 v0.2 specification

Hooks, Gates and Validation



- A hook runs a script when a specified event occurs; the agent need not remember to log it.
- Observation hooks record events. Blocking hooks stop actions only where the host supports it.
- Controller gates decide whether the candidate can advance.

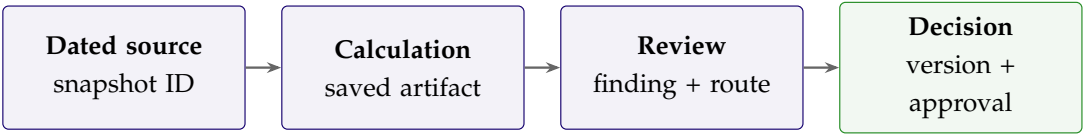
State and Checkpoints



State-Machine Record	What Must Survive a Restart
Identity	Case, request, candidate and prompt-version IDs
Progress	Pending role, round, retry count and deadline
Evidence	File paths and hashes that detect altered artifacts
Findings	Open problems and verified repairs

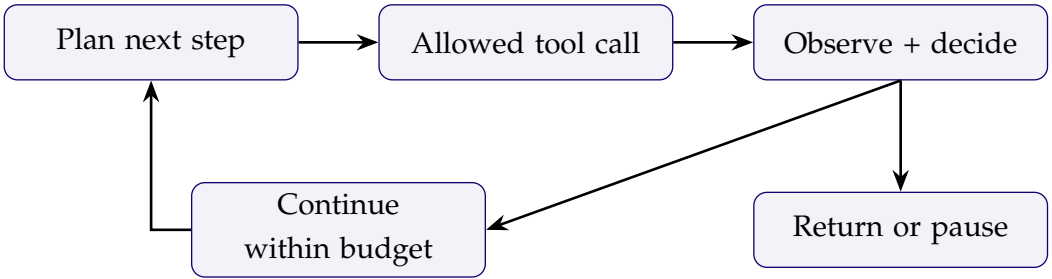
- Save verified state at each transition. A conversation's last line does not show which steps finished.

Evidence and Audit Records



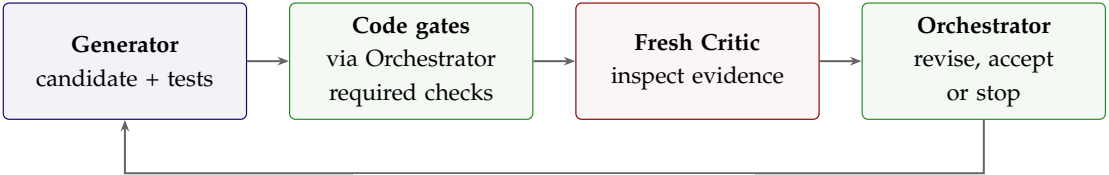
- Save JSON evidence locally and keep Git version history. For each approval, record the approver, timestamp and authorized action or version.
- Another analyst can reconstruct observable actions and files; the record does not reveal hidden model reasoning.

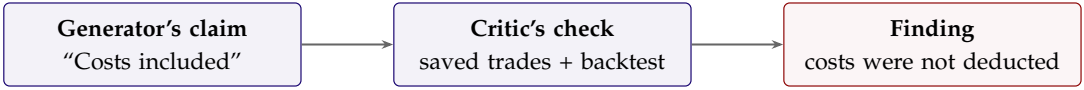
Loop: Act, Observe, Decide



- A failed tool call leaves work to do.
- Use the observation to choose the next action.
- Retry temporary errors within a fixed limit; invalid evidence must not advance the case.

Loop: Generate, Check, Critique





Illustrative review

- Give the Critic saved trades and backtest outputs to inspect.
- A fresh Critic each round can reduce anchoring on earlier responses; shared mistakes remain possible.
- Keep the mandate and acceptance rules fixed during revision.

Loop: Findings and Verified Repairs

Timing Problem

The signal uses day-t closing prices.

The backtest trades at that same close.

The trade must follow signal availability.

Finding Ledger

Round 1: record `pit-001` and link to the affected output.

Round 2 omits it: the finding stays open.

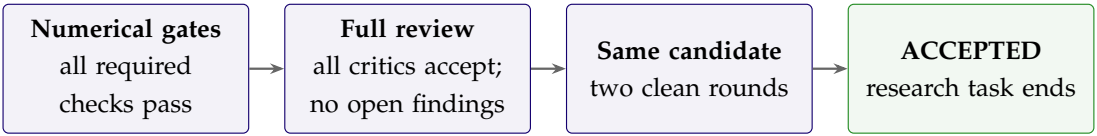
Close only after the Critic verifies the repaired output.

Illustrative

- Question: What check on the saved files would show that the timing problem is fixed?
- Include every unresolved finding in the next request.

Hands up

Loop: Acceptance and Completion



A changed candidate starts a new clean-review streak.

- QT requires two consecutive clean full reviews of the same candidate. A candidate change resets the streak.
- This classroom acceptance rule does not establish profitability or authorize a trade.

Loop: Budgets, Routes, and Stopping

Route / Outcome	Condition
`CONTINUE` (nonterminal route)	Repair or confirmation is possible, with progress and budget remaining.
`REJECTED`	A fatal defect or rejecting Critic verdict ends the candidate.
`BLOCKED`	Required submissions or gates remain invalid after bounded repair.
`EXHAUSTED`	A declared budget is reached before acceptance.
`STALLED`	Unsuccessful rounds repeat without new progress.
`USER_STOPPED`	The user explicitly ends the run.

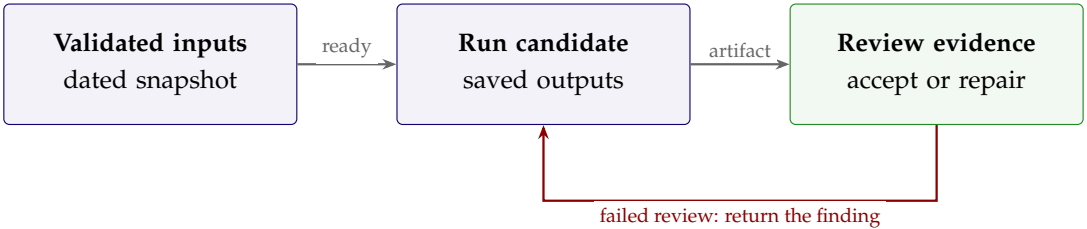
- Set limits on rounds, attempts, tool calls, and elapsed time before starting.
- Record the stopping reason. Exhaustion never turns a failed result into acceptance.

Loop: Recovery and Safe Retry

Observed Problem	Next Check or Action
Timeout	Check whether work completed before a limited retry.
Format or coverage error	Return specific errors to the role that produced the report.
Sealed evidence changed	Block advancement and investigate.
External completion uncertain	Reconcile actual effects before repeating an action.

- Resume only verified pending work; a completed case returns its saved result.
- Reconcile uncertain external actions before retrying to avoid duplicate orders or payments.

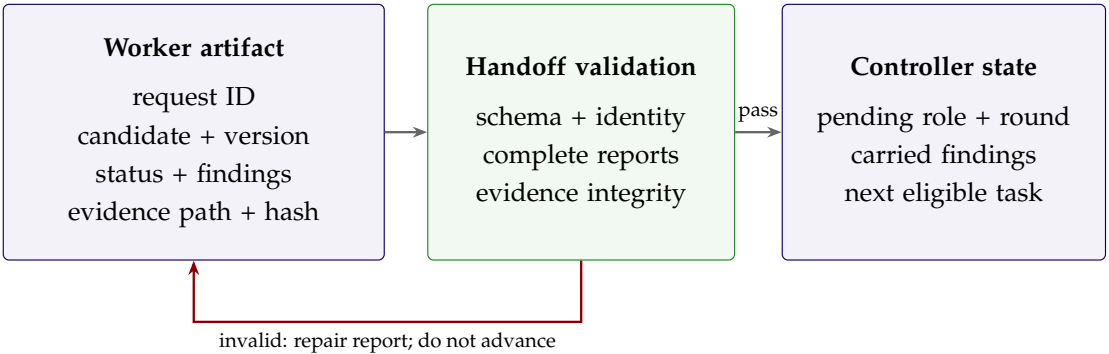
Graph: Dependencies and Handoffs



Missing prerequisites: wait before dispatch.

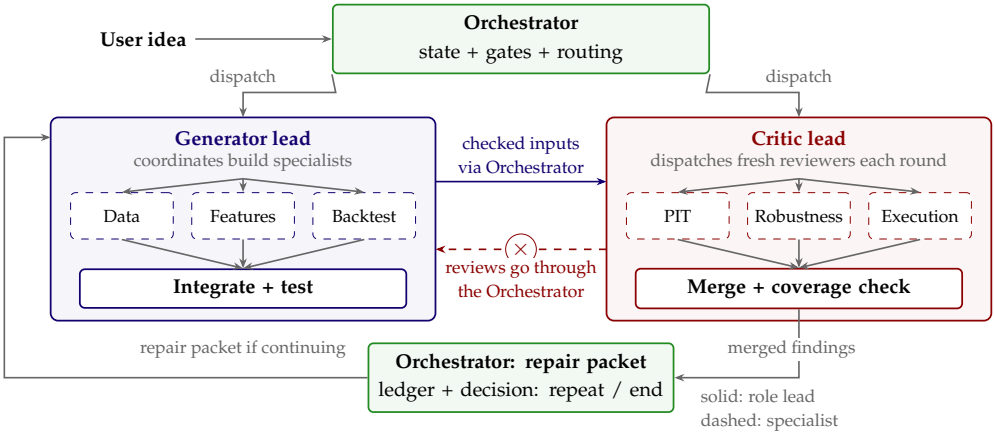
- Nodes perform tasks; edges specify prerequisites and handoffs.
- Wait for missing inputs. Return failed reviews to the Generator.
- A forward chain can be acyclic; its repair edge makes the complete workflow cyclic.

Graph: Artifacts and Shared State



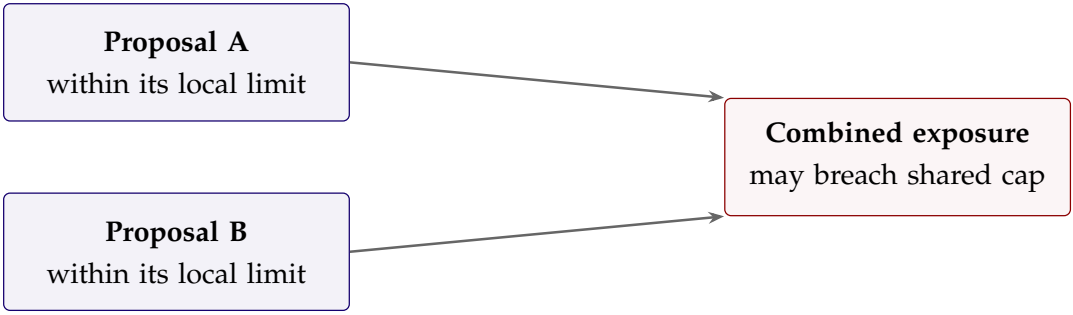
- Workers write separate reports. Only the controller updates the run's control state.
- Validate identity, completeness and evidence before merging.
- Keep conflicting findings explicit; the last report must not silently overwrite them.

Graph: Teams, Fan-Out and Joins



- Use specialists when checks need different expertise.
- Wait for every required report, then check its identity and evidence before combining results.

Graph: Combined Results and Shared Limits



- Two proposals can each pass their own limit and still exceed a shared portfolio limit.
- The classroom fixture does not enforce this portfolio-wide control; it would need to be added.

Graph: Conditional Routing and Recovery

Validated Status	Route	Dependent Work
Inputs missing	Wait for prerequisites	Do not dispatch
Submission invalid	Repair at the same role	Keep downstream blocked
Review needs repair	Return findings to Generator	Recheck revised outputs
Required checks pass	Next eligible task	Join all required reports
Terminal outcome	End and retain the reason	No further dispatch
Authorized input change	Start a new case	Rebuild dependent outputs

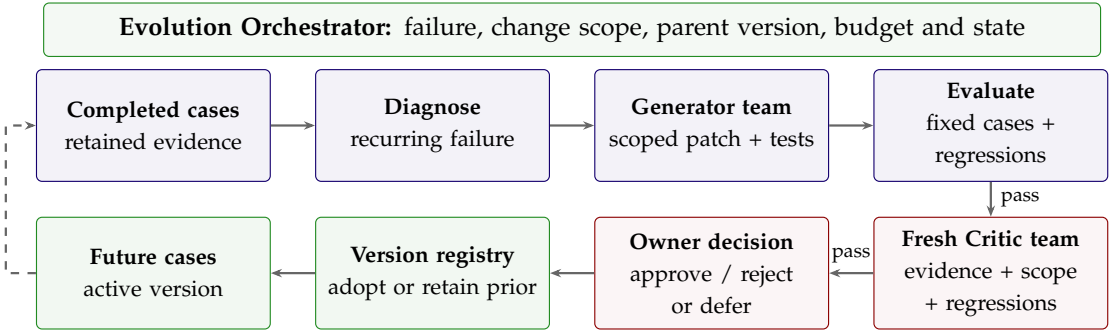
- Choose the next task using validated status and dependencies.
- A changed input version starts a new case: preserve the old evidence and rebuild dependent work.

Evolution: What Can Change?

Change Target	Example	Evidence Needed
Prompt / context	Require missing-evidence disclosure	Behavioral comparison
Skill	Add a source cross-check step	Procedure tests
Tool / code	Return an explicit failure status	Unit and integration tests
Roles / workflow	Add an independent review path	End-to-end evaluation

- The tool loop chooses an action; the research loop revises a candidate.
- Evolution evaluates reusable system changes for adoption, rejection or deferral.
- Strategy-parameter search stays within research. QT demonstrates evolution through Generator-prompt versions.

Evolution: The Detailed Improvement System



- The Generator proposes a scoped change and tests; fresh Critics review evidence, regressions and scope.
- Only versions passing the required checks and review can be approved. Rejected or deferred changes retain the active version.

Evolution: Evidence and Evaluation

Criterion	Required Behavior
Format	Required fields parse correctly.
Meaning	Behavior satisfies the actual task constraint.
Evidence	Cited sources exist in retained evidence and support the claims.
Authority	Unauthorized actions remain blocked.

Development cases
feedback can inform revisions

Held-out cases
separate final assessment

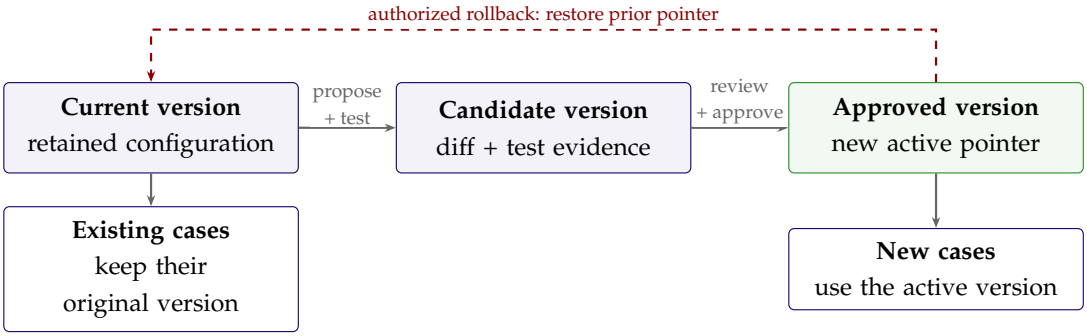
- Set criteria before comparing changes.
- QT timing checks require execution after actual signal availability; a date field alone is insufficient.

Evolution: Regression and Independent Review

What Is Reviewed	Required Evidence
Controller behavior	Missing reports, stale IDs and invalid evidence still block advancement.
Model behavior	Compare old and new outputs against timing and evidence checks.
Change scope	The target and permissions match the approved change request.
Independent review	Check that the change fixes the recorded failure and preserves behavior on reserved cases.

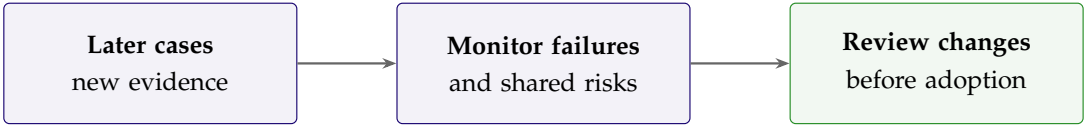
- Code tests cannot establish improved model behavior.
- QT's prompt interface rejects risk-limit, strategy-parameter and controller-code edits.
- Broader changes need separately defined scope and tests; a proposal cannot weaken its own acceptance standard.

Evolution: Versions, Approval and Rollback



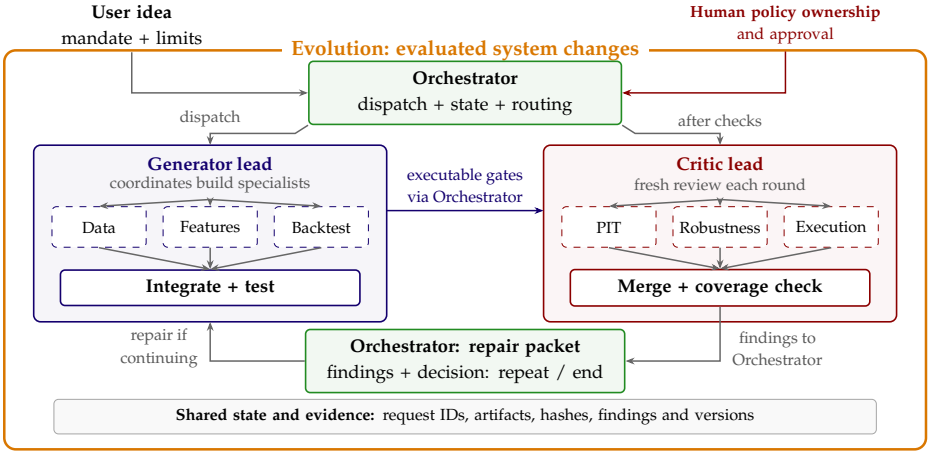
- Record the model, tools, data, policy and prompts tested with each version.
- An alias selects a version; approval needs its own record.
- Retain both versions and evidence. Rollback restores the prior pointer; verify behavior in a new case.

Evolution: Monitoring and Change Triggers



- Monitor recurring failures and shared risks.
- Re-evaluate when models, tools, data feeds or permissions change.
- Feed recorded failures back into evolution. Prompt optimization does not guarantee reliability.

The Complete Architecture: Putting It Together



- Locate the role harnesses, revision loops, handoffs, and evolution boundary.

Roadmap

1. Introduction

2. Design of Long-Run Workflows

3. Case: The Quant System

Build
assemble and run QT
`/qt:start`

Evolve
evaluate a system change
`/qt:evo`

Build: Package and Entry Points

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/{qt,check}.py   # controller actions and test runner
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvements.
- Install with `/plugins install <qt>`. After source edits, reinstall and run `/reload`.
- Keep installed code separate from case state and prompt versions in the workspace's `.qt/` directory.

Build: Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

Source: QT `agents/qt-generator.md` (abbreviated).

- Define the lead's tools and specialist allowlist in its role card.
- Inspect actual host dispatch and response records during a live run.
- Report files alone do not prove that separate specialist contexts ran.

Build: Point-in-Time Inputs

Time	Record	Meaning
2024-03-01	v1: value = 5.0	Initial release available
2024-05-01	v2: value = 4.7	Restated value released
April decision	Use v1 = 5.0	Filter by release date before version

Illustrative

- Select the version publicly available at the decision date.
- A signal calculated from the closing price can only inform a later trade.
- These are the QT data contract's checks; the harness determines where they run and what happens if they fail.

Build: Verified Numerical Tools

Illustrative Worked price example: $12/10 - 1 = 20\%$.

Part	Definition	Implementation
Score	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	21 valid closing prices
Example	$12/10 - 1 = 20\%$	Known-answer unit test
Rank	Descending score; stable tie-break	`baseline/strategy.py`

```
def momentum(closes):
    return closes[-1] / closes[0] - 1
```

Source: QT `baseline/strategy.py`; input validation runs before this function.

- Check dates and price availability before calling the calculation.
- Test the known answer, verify stable ranking, and state the earliest permitted execution time.

Build: The Orchestrator and Gates

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

Source: QT `orchestrator.py`, excerpt from `run_cycle`.

Barrier	Evidence Required
Generator	All assigned specialist reports
Numerical	Executed candidate and passed gates
Critic	Complete review and carried findings
Termination	Acceptance or explicit stop conditions

- Require three matching Generator reports and three fresh Critic reviews.
- Run `python3 <qt>/scripts/check.py` to check the controller rules.

Build: Register and Inspect a Hook

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- Optional hook exercise (not bundled): copy `audit_event.py` into `<qt>/hooks/`, merge its manifest, and reinstall/reload.
- Records event/tool names and session/call IDs once; no candidate assessment or complete specialist trace.
- Kimi logs hook errors/timeouts and continues. Required checks remain in the controller.

Source: Kimi hook behavior

Build: Run One Research Case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
    generator/*.json -> numerical/ -> critic/*.json -> decision.json
  state.json (ledger), trace.jsonl, result.json (terminal round)
```

- Follow the `request_id`, specialist reports, numerical outputs, and routing decision.
- Label scripted rehearsals. Show host dispatch records when claiming a live team ran.
- The final evidence must support acceptance or explain why the case stops.
- The fixture tests momentum candidates; only `lookback` and `top_n` vary.

Build: Repair a Rejected Submission

Submission	Expected Controller Response
One specialist missing	Reject; keep the current role and consume an attempt
Stale request identifier	Same rejection and retry rule
Second invalid submission	`BLOCKED`; no advancement
Complete matching reports	Check evidence before advancing

- Question: Try a missing report and a stale request ID in disposable cases. What repairs allow advancement?
- Call `advance` without `--response`; reread `request.json` and match the new ID across all reports. Check hashes; preserve sealed evidence.

Hands up

Build: Replay, Audit, and Resume

Operation	Evidence or Rule
Reconstruct	Inputs/code; open/repaired findings; stop rule; role card/version; approval evidence, if retained
Recalculate	Freeze inputs, code and configuration; compare numerical results
Fresh model run	New case; retain its own decisions, tool paths and outputs
Resume or reuse	Verify request ID and hashes; resume pending work; a completed case returns its saved result

- Inspect `state.json`, `trace.jsonl`, and saved outputs.
- Start a new case if code, the data snapshot or role instructions change, or for a fresh evaluation.
- Question: Reconstruct this case. Do its files or a separate approval record identify who approved the version?

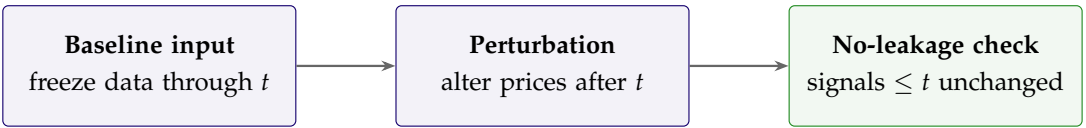
Hands up

Build: Production Data and Adapters

Component	Production Requirement
Inputs	Licensed, dated market and financial data
Universe	Historical index membership, suspensions and delistings
Execution	Lot sizes, price limits, T+1 rules and fees for the relevant board and date
Operations	Host isolation, named responsible people and an independent test of each adapter

- The fixture uses five fictional stocks and synthetic weekday bars, not an exchange calendar or current feed.
- Complete corporate-action, portfolio, and broker adapters remain to be built.

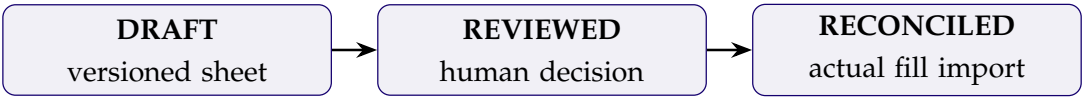
Build: Research Validation



- Test future-price changes, restatements, higher costs, and performance in successive out-of-sample periods.
- Passing these checks does not establish that the strategy will earn excess returns.

Build: Capital Authority and Reconciliation

Target production workflow



- Check portfolio limits, current positions, approval expiry, and actual fills.
- A named person authorizes capital use; research acceptance does not authorize orders.
- `trade_sheet.csv` uses an independent historical synthetic sizing fixture, not the accepted candidate. QT has no broker connection.

Evolve: Capture the Failure

Preparation Step	Evidence or Evaluation Setup
Completed case	Original request and failure artifact
Parent prompt	Exact version before the change
Expected behavior	Observation precedes execution
Case split	Development and untouched final cases

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

- Back in the research workflow, `/qt:evo` starts system improvement; `evo-prepare` links the case and parent prompt version.
- Define the timing behavior before testing. Use development cases for feedback and reserve separate final-evaluation cases.

Evolve: Propose One Prompt Change

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Question: Which instruction addresses the timing omission, and what evidence would show that it worked?
- This QT interface changes the Generator prompt. Strategy parameters, risk limits, acceptance rules, and controller code remain outside its scope.

Hands up

Evolve: Compare Outputs with promptfoo

Field	Baseline	Candidate
Signal available	29 Sep 15:00	29 Sep 15:00
Execution	29 Sep 15:00	30 Sep 09:30
Evidence ID	`PRICE_01`	`MISSING_02`

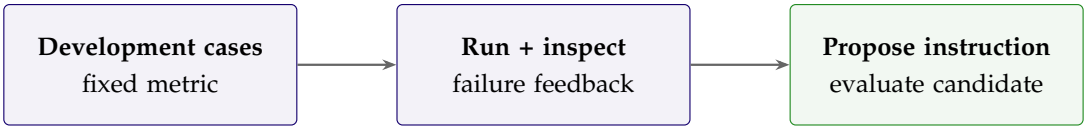
Illustrative

- Trading is allowed from 30 Sep 09:30 (UTC+8). Only `PRICE_01` is retained.
- Run the companion checks, or load `promptfooconfig.json` in promptfoo, to compare both outputs.
- Question: Does either output pass both checks?

Source: promptfoo configuration and assertions; external example

Hands up

Evolve: Generate Candidates with DSPy/GEPA



- GEPA uses a callable program, development cases, a metric, feedback, and a search budget to propose prompt changes.
- Test candidates on held-out cases. A score does not authorize adoption.

Source: DSPy GEPA; external extension, not integrated into QT

Evolve: Judge the Evidence

Check	Baseline	Candidate
Execution timing	Fail	Pass
Evidence reference	Pass	Fail
Adoption decision	Reject	Reject

Illustrative

- The candidate fixes the timing but cites a missing source, `MISSING_02`.
- Question: What must be repaired and independently checked before adopting it?

Hands up

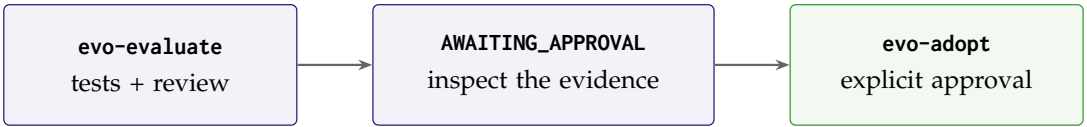
Evolve: Record the Tested Version

Registry	What to Retain
QT local registry	Prompt versions and active pointer
MLflow counterpart	Prompt versions, evaluations, traces
Run configuration	Model, tools, policy, data versions

- QT stores immutable prompt versions and the active pointer in `.qt/`.
- A registry alias selects a version; approval needs its own record.
- MLflow is an external counterpart, not a bundled QT integration.

Source: MLflow Prompt Registry

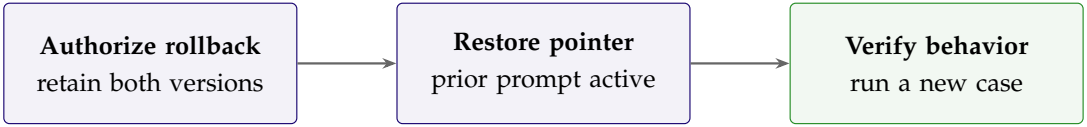
Evolve: Approve or Reject Adoption



- QT checks code regressions and review fields. A person still needs to assess the comparison evidence.
- Question: Would you adopt this version? Explain your decision using the saved results.

Hands up

Evolve: Roll Back and Monitor



- After approval, use `evo-rollback` to restore the previous pointer. Keep both versions and their evidence.
- Existing cases retain their original configuration; they do not switch versions mid-run.
- Verify the restored version in a new case, then monitor later runs for recurring failures.

Review the Completed System

Evidence
trace and reproduce

Control
block or explain

Continuity
resume verified work

Improvement
evaluate and restore

- Question: Choose one control and explain whose need it addresses.
- Show what has been tested, what the results support, and what still needs to be built.

Hands up